Programming Fundamentals

Selection 1 — Decisions with if and if-else

Programming Fundamentals Team

SETU

2026-09-08

Outline

Outline

Outline

Outline

1. Today

By the end of this session you should be able to:

- Explain sequence and selection.
- Recognise Boolean conditions.
- Use relational operators correctly.
- Write `if` and `if-else` statements.
- Trace a program and predict which branch runs.
- Use indentation correctly in Python.

Outline

Outline

2. Warm-up: What happens next?

```python
name = input("Name: ")
print("Hello", name)
print("Welcome to SETU")
```

2. Warm-up: What happens next?

```python
name = input("Name: ")
print("Hello", name)
print("Welcome to SETU")
```

Every statement runs, from top to bottom.

This is sequence: the default flow of execution.

Outline

Outline

3. Flow of Control

Sequence	Statements run in order
Selection	Statements run only when a condition is satisfied
Iteration	Statements are repeated

3. Flow of Control

Sequence	Statements run in order
Selection	Statements run only when a condition is satisfied
Iteration	Statements are repeated

Today we move from programs that always do the same thing to programs that can **make decisions**.

Outline

Outline

4. A Real Decision

A cinema might ask:

```python
age = int(input("Enter your age: "))
```

4. A Real Decision

A cinema might ask:

```python
age = int(input("Enter your age: "))
```

What should happen next?

- If age is 18 or more ⟶ allow entry.
- Otherwise ⟶ refuse entry.

4. A Real Decision

A cinema might ask:

```python
age = int(input("Enter your age: "))
```

What should happen next?

- If age is 18 or more ⟶ allow entry.
- Otherwise ⟶ refuse entry.

The program needs a condition whose answer is either **True** or **False**.

Outline

Outline

5. Boolean Values

Python has a Boolean data type: `bool`.

```
is_raining = True
logged_in = False
```

5. Boolean Values

Python has a Boolean data type: `bool`.

```
is_raining = True
logged_in = False
```

There are only two Boolean values:

- `True`
- `False`

5. Boolean Values

Python has a Boolean data type: `bool`.

```python
is_raining = True
logged_in = False
```

There are only two Boolean values:

- `True`
- `False`

Notice the capital letters.

Outline

Outline

6. Boolean Expressions

A Boolean expression produces `True` or `False`.

```
age >= 18
mark >= 40
password == "python"
number < 0
```

6. Boolean Expressions

A Boolean expression produces `True` or `False`.

```
age >= 18
mark >= 40
password == "python"
number < 0
```

Try these in the Python REPL:

```
>>> 10 > 5
True
>>> 10 == 5
False
```

Outline

Outline

7. Relational Operators

Operator	Example	Meaning
`>`	`x > y`	greater than
`>=`	`x >= y`	greater than or equal to
`<`	`x < y`	less than
`<=`	`x <= y`	less than or equal to
`==`	`x == y`	equal to
`!=`	`x != y`	not equal to

Outline

Outline

8. = versus ==

```
age = 18
```

means **put 18 into age**.

8. = versus ==

```
age = 18
```

means **put 18 into age**.

```
age == 18
```

asks **is age equal to 18?**

8. = versus ==

```
age = 18
```

means **put 18 into age**.

```
age == 18
```

asks **is age equal to 18?**

> **Common beginner error:** = assigns; == compares.

Outline

Outline

9. Predict the Result

What does each expression produce?

```
10 > 3
7 == 7
5 != 5
12 <= 10
"cat" == "cat"
```

9. Predict the Result

What does each expression produce?

```
10 > 3
7 == 7
5 != 5
12 <= 10
"cat" == "cat"
```

Answers: `True`, `True`, `False`, `False`, `True`.

Outline

Outline

10. The if Statement

```
if condition:
    statement
```

10. The if Statement

```
if condition:
    statement
```

Example:

```
age = 20

if age >= 18:
    print("You are an adult")
```

10. The if Statement

```
if condition:
    statement
```

Example:

```
age = 20

if age >= 18:
    print("You are an adult")
```

The indented statement runs **only if** the condition is `True`.

Outline

Outline

11. Indentation Matters

Correct:

```python
if mark >= 40:
    print("Pass")
    print("Well done")
```

11. Indentation Matters

Correct:

```python
if mark >= 40:
    print("Pass")
    print("Well done")
```

Incorrect:

```python
if mark >= 40:
print("Pass")
```

> Python uses indentation to define the block belonging to the `if`.

Outline

Outline

12. Trace an if

```python
temperature = 25

if temperature > 20:
    print("Warm day")

print("Finished")
```

12. Trace an if

```python
temperature = 25

if temperature > 20:
    print("Warm day")

print("Finished")
```

Output:

```
Warm day
Finished
```

12. Trace an if

```python
temperature = 25

if temperature > 20:
    print("Warm day")

print("Finished")
```

Output:

```
Warm day
Finished
```

What changes if `temperature = 12`?

Outline

Outline

13. One-way Selection

An `if` statement is **one-way selection**.

```
balance = 100
withdrawal = 40

if withdrawal <= balance:
    balance = balance - withdrawal

print(balance)
```

13. One-way Selection

An `if` statement is **one-way selection**.

```
balance = 100
withdrawal = 40

if withdrawal <= balance:
    balance = balance - withdrawal

print(balance)
```

If the condition is false, Python simply skips the indented block.

Outline

Outline

14. Mini Activity: Free Delivery

An online shop gives free delivery when an order is €50 or more.

Complete:

```
order_total = float(input("Order total: €"))

# your selection here
```

14. Mini Activity: Free Delivery

An online shop gives free delivery when an order is €50 or more.

Complete:

```
order_total = float(input("Order total: €"))

# your selection here
```

Write the condition first. Then decide what statement belongs inside the block.

Outline

Outline

15. Activity Solution

```python
order_total = float(input("Order total: €"))

if order_total >= 50:
    print("Free delivery")
```

15. Activity Solution

```python
order_total = float(input("Order total: €"))

if order_total >= 50:
    print("Free delivery")
```

Question: what happens for an order of €30?

Outline

Outline

16. When We Need Two Outcomes

Suppose every mark must be classified as either:

- Pass
- Fail

16. When We Need Two Outcomes

Suppose every mark must be classified as either:

- Pass
- Fail

We could write two separate `if` statements...

```
if mark >= 40:
    print("Pass")
if mark < 40:
    print("Fail")
```

But Python gives us a clearer structure.

Outline

Outline

17. if-else

```
if condition:
    statements_when_true
else:
    statements_when_false
```

17. if-else

```
if condition:
    statements_when_true
else:
    statements_when_false
```

Exactly **one** branch runs.

Outline

Outline

18. Pass or Fail

```python
mark = int(input("Enter mark: "))

if mark >= 40:
    print("Pass")
else:
    print("Fail")
```

18. Pass or Fail

```python
mark = int(input("Enter mark: "))

if mark >= 40:
    print("Pass")
else:
    print("Fail")
```

Trace it for:

- `mark = 72`
- `mark = 38`
- `mark = 40`

Outline

Outline

19. Boundary Values Matter

Consider:

```python
if age > 18:
    print("Adult")
else:
    print("Not adult")
```

19. Boundary Values Matter

Consider:

```python
if age > 18:
    print("Adult")
else:
    print("Not adult")
```

What happens when `age == 18`?

19. Boundary Values Matter

Consider:

```
if age > 18:
    print("Adult")
else:
    print("Not adult")
```

What happens when `age == 18`?

If 18 counts as adult, the condition should be:

```
if age >= 18:
```

Always test values **at the boundary**.

Outline

Outline

20. Strings in Conditions

Selection is not limited to numbers.

```
answer = input("Continue? y/n: ")

if answer == "y":
    print("Continuing...")
else:
    print("Stopping...")
```

20. Strings in Conditions

Selection is not limited to numbers.

```
answer = input("Continue? y/n: ")

if answer == "y":
    print("Continuing...")
else:
    print("Stopping...")
```

String comparisons are case-sensitive: `"Y" != "y"`.

Outline

Outline

21. Making String Input Friendlier

```python
answer = input("Continue? y/n: ").lower()

if answer == "y":
    print("Continuing...")
else:
    print("Stopping...")
```

21. Making String Input Friendlier

```
answer = input("Continue? y/n: ").lower()

if answer == "y":
    print("Continuing...")
else:
    print("Stopping...")
```

Now `Y` and `y` are both treated as `"y"`.

Outline

Outline

22. Common Errors

Which errors can you spot?

```
age = input("Age: ")

if age = 18
    print("You are 18")
else
print("You are not 18")
```

22. Common Errors

Which errors can you spot?

```
age = input("Age: ")

if age = 18
    print("You are 18")
else
print("You are not 18")
```

Problems:

- `input()` returns a string.
- `=` should be `==` for comparison.
- `:` is missing.
- indentation is missing.

Outline

Outline

23. Fix the Program

Correct version:

```
age = int(input("Age: "))

if age == 18:
    print("You are 18")
else:
    print("You are not 18")
```

Outline

Outline

24. Mini-program: Ticket Price

Requirement:

- Under 16 $\longrightarrow$ €8
- Everyone else $\longrightarrow$ €12

24. Mini-program: Ticket Price

Requirement:

- Under 16 ⟶ €8
- Everyone else ⟶ €12

```
age = int(input("Age: "))

if age < 16:
    price = 8
else:
    price = 12

print("Ticket price: €", price)
```

Outline

Outline

25. Think Before You Code

For each problem, identify:

1. What data do I need?
2. What is the Boolean condition?
3. What happens when it is true?
4. Do I need an `else`?

25. Think Before You Code

For each problem, identify:

1. What data do I need?
2. What is the Boolean condition?
3. What happens when it is true?
4. Do I need an `else`?

This planning habit becomes increasingly important as selection gets more complex.

Outline

Outline

26. Practice Challenge

Write a program that asks for a number.

- If it is positive, print `Positive`.
- Otherwise print `Zero or negative`.

26. Practice Challenge

Write a program that asks for a number.

- If it is positive, print `Positive`.
- Otherwise print `Zero or negative`.

Extension: how could you distinguish between **zero** and **negative**?

That leads us into the next session.

Outline

Outline

27. Session 1 Recap

You should now be comfortable with:

- Boolean values and conditions
- `>`, `<`, `>=`, `<=`, `==`, `!=`
- `if`
- `if-else`
- indentation
- tracing branches
- testing boundary values

Next: **more than two choices** and **combining conditions**.